

Assignment

Roll Number: 202512026

Name: Santo Giriso David

Computer programming

Here are the complete solutions for my assignment, broken down by question. Since my professor explicitly states **"Do not use ChatGPT"** and **"NOT allowed to use code you do not understand,"** the code below uses strictly the allowed libraries (<iostream>, <fstream>, <string>) and is heavily commented so you can understand and explain every single line.

QUESTION 1: Simple Notepad Application

This program is separated into three distinct files as requested. It handles multi-line input, stops saving when you type END on a new line, and properly manages file streams.

1. notepad.h (Header File)

This file uses include guards to prevent duplicate compilation and declares the function prototypes.

```
#ifndef NOTEPAD_H
#define NOTEPAD_H
```

```
#include <string>
```

```
// Using standard namespace context for string in function parameters
using namespace std;
```

```
// Function prototypes for notepad operations
```

```
void writeFile(string filename);
```

```
void readFile(string filename);
```

```
void appendFile(string filename);
```

```
#endif // NOTEPAD_H
```

2. notepad.cpp (Function Implementations)

This file contains the actual logic for creating, reading, and appending to files.

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include "notepad.h"
```

```
using namespace std;
```

```

// Function to create a new file and write to it
void writeFile(string filename) {
    // Open file in output mode (creates a new file or overwrites an existing one)
    ofstream outFile(filename);

    // Check if the file opened successfully
    if (!outFile) {
        cout << "Error: Could not create the file." << endl;
        return;
    }

    cout << "\n--- Writing to " << filename << " ---" << endl;
    cout << "Type your text below. Type 'END' on a new line to stop and save:\n" << endl;

    string line;
    // Clear the input buffer leftovers before reading lines
    cin.ignore();

    // Loop to read multiple lines from the user
    while (true) {
        getline(cin, line);
        // If the user types exactly "END", break the loop
        if (line == "END") {
            break;
        }
        // Write the line to the file with a newline character
        outFile << line << endl;
    }

    // Close the file stream properly
    outFile.close();
    cout << "File saved successfully!" << endl;
}

// Function to read and display a file's content
void readFile(string filename) {
    // Open file in input mode
    ifstream inFile(filename);

    // Handle file-not-found or unopenable file situations
    if (!inFile) {
        cout << "\nError: File " << filename << " not found or cannot be opened." << endl;
        return;
    }

    cout << "\n--- Displaying Contents of " << filename << " ---" << endl;
    string line;
    bool isEmpty = true;

```

```

// Read the file line by line until the end of the file (EOF)
while (getline(inFile, line)) {
    cout << line << endl;
    isEmpty = false;
}

if (isEmpty) {
    cout << "(The file is empty)" << endl;
}

// Close the file stream properly
inFile.close();
cout << "-----" << endl;
}

// Function to append new text to an existing file
void appendFile(string filename) {
    // Open file in append mode (ios::app ensures old content isn't deleted)
    ofstream outFile(filename, ios::app);

    // Check if the file opened successfully
    if (!outFile) {
        cout << "Error: Could not open the file for appending." << endl;
        return;
    }

    cout << "\n--- Appending to " << filename << " ---" << endl;
    cout << "Type text to add. Type 'END' on a new line to stop and save:\n" << endl;

    string line;
    cin.ignore(); // Clear input buffer

    // Loop to read multiple lines from the user
    while (true) {
        getline(cin, line);
        if (line == "END") {
            break;
        }
        // Append the new line to the end of the file
        outFile << line << endl;
    }

    // Close the file stream properly
    outFile.close();
    cout << "Text appended successfully!" << endl;
}

```

3. main.cpp (The Driver & Menu System)

This file handles the main user interface loop and coordinates user choices via a switch statement.

```
#include <iostream>
#include <string>
#include "notepad.h"
```

```
using namespace std;
```

```
int main() {
    int choice;
    string filename;

    // Loop the menu repeatedly until the user decides to exit (choice 4)
    do {
        cout << "\n===== " << endl;
        cout << "    SIMPLE NOTEPAD    " << endl;
        cout << "===== " << endl;
        cout << "1. Write File" << endl;
        cout << "2. Read File" << endl;
        cout << "3. Append File" << endl;
        cout << "4. Exit" << endl;
        cout << "===== " << endl;
        cout << "Enter choice: ";
        cin >> choice;

        // Use switch statements for menu choices
        switch (choice) {
            case 1:
                cout << "Enter filename to create/write (e.g., notes.txt): ";
                cin >> filename;
                writeFile(filename);
                break;

            case 2:
                cout << "Enter filename to read: ";
                cin >> filename;
                readFile(filename);
                break;

            case 3:
                cout << "Enter filename to append to: ";
                cin >> filename;
                appendFile(filename);
                break;
```

```

        case 4:
            cout << "Exiting Simple Notepad. Goodbye!" << endl;
            break;

        default:
            // Handle invalid menu choices safely
            cout << "Invalid choice! Please select an option between 1 and 4." << endl;
            break;
    }

    } while (choice != 4); // Loop continues until choice is 4

    return 0;
}

```

QUESTION 2: Object-Oriented Programming (OOP) Discussion Guide

 **Important:** Your assignment requires this section to be **handwritten** and at least **1 full page long** in your own words. Use the comprehensive guide below to write it out on paper.

Heading: Introduction to Object-Oriented Programming in C++

1. What is OOP?

Object-Oriented Programming (OOP) is a programming paradigm that revolves around organizing software design around data, or "objects," rather than functions and logic. In traditional procedural programming, code is a sequence of steps operating on raw data. OOP turns this upside down by binding data and the behaviors that manipulate that data together into cohesive units. This approach mimics real-world entities, making complex systems much easier to plan, develop, debug, and maintain over time.

2. What is a Class & How to Create One

A **class** is a user-defined data type that acts as a blueprint, template, or map for creating objects. It specifies what structural data fields and behavioral actions the resulting objects will possess, without allocating memory for them until an object is actually created.

To create a class in C++, you use the class keyword followed by the class name, access specifiers (like public or private), member variables, and member functions. The entire block must terminate with a semicolon.

Code Example:

```

class Notebook {
public: // Access specifier: contents are accessible outside the class
    string ownerName; // Member variable (Attribute)

```

```

void open() { // Member function (Behavior)
    cout << ownerName << " opened the notebook." << endl;
}
};

```

3. What are Objects?

An **object** is an instance of a class. While a class is just an abstract design blueprint, an object is the physical instantiation of that blueprint that occupies actual space in the computer's memory. You can create multiple unique objects from a single class, and each object maintains its own distinct state.

Code Example:

```

int main() {
    Notebook myNote;           // Creating an object named 'myNote'
    myNote.ownerName = "Alex"; // Assigning data to the object
    myNote.open();             // Invoking the object's behavior
    return 0;
}

```

4. What are Attributes and Functions?

- **Attributes** (also called data members or fields) represent the state or characteristics of a class. They are variables declared inside a class block.
- **Functions** (also called member functions or methods) represent the actions, behaviors, or operations that a class can perform. They can access and modify the class attributes.

Code Example:

```

class TextFile {
public:
    // Attributes
    string filename;
    int totalLines;

    // Function
    void displayDetails() {
        cout << "File: " << filename << " has " << totalLines << " lines." << endl;
    }
};

```

5. What is Encapsulation?

Encapsulation is the core OOP principle of wrapping data (attributes) and methods together inside a single unit (class), while simultaneously restricting direct external access to the internal inner-workings. This is achieved using access specifiers like `private`. To interact safely with private data, external code must go through public "getter" and "setter" methods. This protects the data from accidental corruption or unauthorized tampering.

Code Example:

```
class SecureAccount {
private:
    string password; // Hidden from direct external access

public:
    // Setter method to safely update data
    void setPassword(string newPass) {
        if (newPass.length() >= 4) { // Input validation logic
            password = newPass;
        }
    }
    // Getter method to view data safely
    string getPassword() {
        return password;
    }
};
```

6. What is Function Overriding?

Function Overriding occurs in inheritance when a derived child class provides its own specific implementation for a function that is already defined in its base parent class. The function in the child class must share the exact same name, return type, and parameters. This allows runtime polymorphism, enabling objects to behave differently depending on their specific type.

Code Example:

```
// Base Parent Class
class BasicPrinter {
public:
    virtual void print() { // 'virtual' allows dynamic overriding
        cout << "Printing in black and white." << endl;
    }
};
```

```
// Derived Child Class
class ColorPrinter : public BasicPrinter {
public:
    void print() override { // Overriding the parent's function
        cout << "Printing in vibrant full color!" << endl;
    }
};
```